Billetera Virtual

Proyecto PCII 2026 — Primera etapa: análisis y diseño de la solución
Integrantes: [completar, máx. 3] · Lenguaje: C (C11)

1. Análisis del problema

Se necesita simular una billetera virtual: se registran usuarios, cada uno con un saldo y un historial de operaciones (ingresos y egresos), y se pueden consultar y listar según distintos criterios. No hay transacciones reales: el objetivo es gestionar y consultar la información en memoria.

Aspecto	Análisis
Datos de un usuario	identificador único, nombre, saldo disponible, conjunto de operaciones realizadas.
Datos de una operación	identificador, tipo (ingreso / egreso), monto, fecha.
Volumen	Cantidad de usuarios y de operaciones **desconocida y variable**: se agregan y se eliminan usuarios durante la ejecución. Las operaciones solo se agregan (el historial no se edita ni se borra).
Operaciones requeridas	Registrar usuario, buscar por id, eliminar usuario, ingreso, egreso, registrar operación y actualizar saldo, consultar saldo, y listar usuarios según un criterio. Se implementan las 7 (el mínimo pedido es 4) más el listado.
Operaciones más frecuentes	Ingreso, egreso, consulta de saldo y búsqueda: todas necesitan **localizar un usuario por id**. Por eso la búsqueda por id es la operación que más pesa en la elección de la estructura.
Recorridos	El listado por criterio recorre todos los usuarios; el historial de un usuario se recorre en orden de registro.

Supuestos y reglas de negocio

- El **id de usuario lo asigna el sistema** (1, 2, 3, …) y nunca se reutiliza. Esto garantiza unicidad y es un dato clave para la elección de estructura (ver 3.1).
- Los **montos se guardan en centavos** como entero (`long long`), no como `float`, para evitar errores de redondeo en dinero.
- El monto debe ser mayor que cero; la fecha debe existir en el calendario (se validan años bisiestos); el nombre no puede ser vacío.
- Un egreso solo se acepta si `monto ≤ saldo`; el saldo nunca queda negativo.
- **Registrar la operación y actualizar el saldo es un único paso** (operaciones 4, 5 y 6 del enunciado): no existe forma de modificar el saldo sin dejar la operación en el historial, de modo que el saldo siempre es la suma de ingresos menos egresos.
- Un usuario solo puede eliminarse con **saldo cero**. Es nuestra interpretación de “contemplar adecuadamente la información asociada”: evita “destruir” dinero. Al eliminarlo se libera también todo su historial.
- Cada operación recibe un id global creciente que asigna la billetera.

2. Descomposición top-down

```
Billetera Virtual
├─ 1. Interfaz de usuario (main)
│    ├─ 1.1 Menú y selección de opción
│    ├─ 1.2 Lectura y validación de entradas (id, monto, fecha, texto)
│    └─ 1.3 Presentación de resultados y mensajes de error
├─ 2. Gestión de usuarios  (TAD Wallet)
│    ├─ 2.1 Registrar usuario        (asignar id, insertar)
│    ├─ 2.2 Buscar usuario por id
│    └─ 2.3 Eliminar usuario         (verificar saldo 0, quitar, liberar historial)
├─ 3. Gestión de movimientos  (TAD Wallet + TAD OperationList)
│    ├─ 3.1 Validar monto / fecha / saldo suficiente
│    ├─ 3.2 Registrar operación en el historial
│    ├─ 3.3 Actualizar saldo
│    └─ 3.4 Consultar saldo
├─ 4. Listado de usuarios por criterio
│    ├─ 4.1 Recorrer todos los usuarios en orden de id
│    └─ 4.2 Aplicar filtro (saldo > X / operaciones en un período / más de N)
└─ 5. Utilitarios de fecha (TAD Date): validar, comparar, fecha actual
```

3. Elección de estructuras de datos

Lema de diseño: *¿qué estructura representa bien la información y permite hacer eficientemente las operaciones que la aplicación necesita?* Con n = cantidad de usuarios y k = cantidad de operaciones de un usuario:

3.1 Colección de usuarios

Estructura	Buscar por id	Insertar	Eliminar	Recorrido en orden de id	Comentario
Lista enlazada no ordenada	O(n)	O(1)	O(n)	requiere ordenar	La búsqueda domina el costo de casi toda la aplicación.
Lista enlazada ordenada	O(n)	O(n)	O(n)	O(n)	No mejora la búsqueda.
Arreglo ordenado	O(log n)	O(n)	O(n)	O(n)	Corrimiento de elementos al insertar y eliminar.
ABB sin balanceo	O(log n) prom. O(n) peor	ídem	ídem	O(n)	**Con ids crecientes degenera en una lista** (cada alta va a la derecha): O(n) en el caso normal de uso.
Árbol AVL por id	**O(log n)**	**O(log n)**	**O(log n)**	**O(n)**	Peor caso garantizado: altura ≤ $1{,}44 \cdot \log_2(n+2)$. Recorrido in-order entrega los usuarios ordenados por id.

Decisión: árbol AVL ordenado por id. Como el sistema asigna ids crecientes, un ABB común quedaría con forma de lista y perdería su ventaja; el AVL se rebalancea con rotaciones en cada inserción y eliminación y mantiene O(log n) en el peor caso. El costo es la complejidad extra de implementación (altura en cada nodo, 4 casos de rotación).

No se eligió una tabla hash porque el listado necesita recorrer los usuarios en un orden determinado y porque el AVL da cota de peor caso sin tener que redimensionar.

3.2 Historial de operaciones de cada usuario

Estructura	Agregar al final	Acceso a la i-ésima	Recorrer	Comentario
Lista enlazada simple (con puntero al último)	O(1)	O(i)	O(k)	Un puntero por nodo y una reserva de memoria por operación.
Arreglo dinámico (capacidad inicial 4, se duplica)	**O(1) amortizado**	**O(1)**	**O(k)**	Memoria contigua, pocas reservas. Alcanza porque **nunca se eliminan operaciones sueltas**: solo se agregan y se consulta.

Decisión: arreglo dinámico. Las únicas operaciones sobre el historial son agregar al final, recorrerlo y contarlo; el arreglo cumple las tres con el menor costo. La cantidad de operaciones queda disponible en O(1), lo que simplifica el criterio “más de N operaciones”.

4. Costo de las operaciones de la aplicación

Operación	Algoritmo	Costo (peor caso)
Registrar usuario	Asignar id, crear nodo e insertar en el AVL (recursivo)	O(log n)
Buscar usuario por id	Descenso por el árbol (recursivo)	O(log n)
Ingreso / Egreso (registrar y actualizar saldo)	Buscar + validar + agregar al arreglo + actualizar saldo	O(log n) + O(1) amortizado
Consultar saldo	Buscar; el saldo se mantiene actualizado en el usuario	O(log n)
Eliminar usuario	Buscar, verificar saldo 0, quitar del AVL con reemplazo por el sucesor in-order (recursivo), liberar historial	O(log n + k)
Listar: saldo > X	Recorrido in-order con filtro O(1) por usuario	O(n)
Listar: más de N operaciones	Recorrido in-order, compara la cantidad guardada	O(n)
Listar: con operaciones en un período	Recorrido in-order; por usuario busca una fecha dentro del rango (corta al primer acierto)	O(n + total de operaciones)
Listar todos	Recorrido in-order, sale ordenado por id	O(n)

> **¿Por qué el listado es O(n) y no se indexa por saldo?** Los criterios de listado no coinciden con la clave del árbol (id). Un segundo índice por saldo o por cantidad de operaciones habría que reubicarlo en *cada* ingreso y egreso (O(log n) extra en las operaciones más frecuentes) para acelerar una consulta ocasional. Como el listado igualmente tiene que mostrar los resultados (que pueden ser todos), se prefiere un único recorrido lineal y una estructura simple.

5. Algoritmos recursivos

La estructura es un árbol, así que la recursión es la forma natural de expresar las operaciones (la rama izquierda y la derecha son subproblemas del mismo tipo). Se resuelven recursivamente:

- `insert_node`: inserta en el subárbol que corresponde y rebalancea al volver. Recurrencia T(h) = T(h−1) + O(1) ⇒ O(log n).
- `find_node`: descarta un subárbol por llamada ⇒ O(log n).
- `remove_node` / `remove_min`: baja hasta el nodo, lo reemplaza por el mínimo del subárbol derecho y rebalancea el camino ⇒ O(log n).
- `for_each_node`: recorrido in-order (izquierdo, nodo, derecho); visita cada usuario una vez, ordenado por id ⇒ O(n).
- `destroy_nodes`: recorrido post-order que libera hijos antes que el nodo y su historial ⇒ O(n + total de operaciones).

6. Especificación de los TAD

Se identifican cinco TAD. Los usuarios del TAD solo conocen su interfaz (`.h`); las operaciones están pensadas sin depender de la representación elegida.

6.1 TAD Date (interfaz)

Operación	**Descripción**
`int date_is_valid(Date d)`	Verdadero si la fecha existe en el calendario gregoriano (1900–2100).
`int date_compare(Date a, Date b)`	Negativo, cero o positivo según a sea anterior, igual o posterior a b.
`Date date_today(void)`	Fecha actual del sistema.

6.2 TAD Operation (interfaz)

Registro con `id`, `type` (DEPOSIT o WITHDRAWAL), `amount` (centavos) y `date`. Es un dato simple de solo lectura una vez creado.

6.3 TAD OperationList (interfaz) **— historial de un usuario**

Operación	**Descripción**
`operation_list_create()`	Crea una lista vacía.
`operation_list_append(list, op)`	Agrega `op` al final. O(1) amortizado.
`operation_list_count(list)`	Cantidad de operaciones. O(1).
`operation_list_get(list, pos)`	Operación en la posición `pos`, o NULL si está fuera de rango. O(1).
`operation_list_destroy(list)`	Libera la lista y sus operaciones.

6.4 TAD User (interfaz)

Agrupa `id`, `name`, `balance` (centavos) y el historial (`OperationList`). Invariante: `balance` = suma de ingresos − suma de egresos del historial, y `balance ≥ 0`.

6.5 TAD Wallet — implementado en esta entrega

Conjunto de usuarios indexados por id. Es el TAD central: concentra las operaciones del enunciado y es el que se implementa en detalle (sección 7).

Operación	Precondición → Postcondición
`wallet_create()`	— → billetera vacía.
`wallet_register(w, name, &id)`	nombre no vacío → nuevo usuario con saldo 0, historial vacío e id único asignado.
`wallet_find(w, id)`	— → puntero (solo lectura) al usuario, o NULL si no existe.
`wallet_remove(w, id)`	usuario existente y con saldo 0 → usuario e historial eliminados. Si no existe o tiene saldo devuelve el código de error correspondiente y no modifica nada.
`wallet_deposit(w, id, monto, fecha)`	monto > 0, fecha válida, usuario existente → saldo += monto y operación registrada.
`wallet_withdraw(w, id, monto, fecha)`	ídem y monto ≤ saldo → saldo −= monto y operación registrada. Si no hay saldo suficiente, no se modifica nada.
`wallet_for_each(w, accion, datos)`	— → aplica `accion` a cada usuario en orden creciente de id. Es la base de los listados: el criterio lo decide quien llama, el TAD no conoce los filtros.
`wallet_count(w)`	— → cantidad de usuarios.
`wallet_destroy(w)`	libera la billetera completa.

Las operaciones que pueden fallar devuelven un código `Result` (OK, NOT_FOUND, INSUFFICIENT_FUNDS, BALANCE_NOT_ZERO, INVALID_AMOUNT, INVALID_DATE) en lugar de imprimir: la lógica queda separada de la interfaz de usuario y se puede probar por separado.

Decisión de encapsulamiento: `Wallet` y `OperationList` son tipos opacos (el `.h` solo declara el nombre; la estructura vive en el `.c`), por lo que el programa principal no depende del AVL ni del arreglo dinámico. Si se cambiara la estructura interna, `main.c` no se modifica. Los usuarios se devuelven como `const User *` para que desde afuera no se pueda alterar el saldo salteándose las validaciones.

7. Implementación del TAD Wallet (decisiones)

- **Representación:** nodo AVL con `User`, `height` y punteros `left`/`right`; la billetera guarda la raíz, la cantidad de usuarios y los contadores de ids de usuario y de operación.
- **Invariante AVL:** para todo nodo, |altura(izq) − altura(der)| ≤ 1. Se restablece con `rebalance` al volver de cada llamada recursiva (rotación simple o doble según el caso).
- **Eliminación:** si el nodo tiene dos hijos se lo reemplaza por el sucesor (mínimo del subárbol derecho), que se extrae con `remove_min`; luego se rebalancea el camino recorrido.
- **Alta:** el id sale de un contador interno que solo crece, de modo que ids de usuarios eliminados no se reutilizan.
- **Memoria:** `wallet_destroy` y `wallet_remove` liberan el historial de cada usuario; se verificó la ejecución con AddressSanitizer / UBSan sin errores.

8. Criterio de listado

El menú ofrece tres criterios sobre el mismo recorrido `wallet_for_each`: **saldo mayor que un monto**, **con operaciones dentro de un período** y **con más de N operaciones**. Se separa el *recorrido* (responsabilidad del TAD) del *filtro* (responsabilidad de la aplicación), de modo que agregar otro criterio no requiere tocar el TAD. Los resultados salen ordenados por id.

9. Plan de pruebas

Caso	Resultado esperado
Registrar usuarios, buscar uno existente y uno inexistente	Ids 1, 2, 3…; el inexistente informa “No existe un usuario con ese id”.
Ingreso de $100,50, egreso de $30,00, consultar saldo	Saldo $70,50; 2 operaciones en el historial.
Egreso mayor al saldo	“Saldo insuficiente”; saldo e historial sin cambios.
Monto 0 o negativo; fecha 31/02 o 29/02 en año no bisiesto	Rechazados con mensaje.
Eliminar usuario con saldo / sin saldo	Con saldo: rechazado. Sin saldo: eliminado junto con su historial.
Alta de muchos usuarios consecutivos (p. ej. 1000)	El árbol se mantiene balanceado (altura ≈ $\log_2$ n), no con forma de lista.
Listados por cada criterio, con 0 usuarios y con resultados vacíos	Resultados correctos y mensaje cuando no hay coincidencias.
Ejecución con AddressSanitizer	Sin pérdidas ni accesos inválidos.

10. Limitaciones y posibles mejoras

- El historial se guarda en **orden de registro**, no necesariamente cronológico: el usuario ingresa la fecha de cada operación y podría cargar una anterior a la última. Para el alcance del enunciado no afecta los criterios de listado.
- No hay persistencia: los datos se pierden al cerrar el programa.
- No se contempla la falta de memoria (`malloc`/`realloc` devolviendo NULL).
- El criterio “con operaciones en un período” es lineal en el historial de cada usuario; con historiales muy grandes y fechas ordenadas se podría usar búsqueda binaria.

11. Entrega

Código fuente y ejecutable: [pegar enlace a la carpeta / online GDB]

Archivos: `main.c` (interfaz), `wallet.c/.h` (TAD Wallet, AVL), `operation_list.c/.h`, `operation.h`, `user.h`, `date.c/.h`, `Makefile`.